

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What happens when this code runs?

```
def first():
    second()
```

```
def second():
    third()
```

```
def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

- A)NameError
- B)Handled
- C)TypeError
- D)RuntimeError cannot propagate through functions

Q2. What happens when this function is called?

```
def fun(a=10, b):
    return a + b
```

- A)TypeError
- B)It executes correctly
- C)NameError
- D)SyntaxError

Q3. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
```

```
sample.display()
```

- A)Error
- B)x
- display
- C)10
- Hello
- D)Only Hello

Q4. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

- A)10 followed by Finally
- B)Finally followed by 10
- C)Finally only
- D)10 only

Q5. What happens here?

```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

- A)Wrong Value
- B)e
- C)ValueError
- D)Nothing

Q6. What is the output?

```
def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
```



```
finally:  
    print("D")
```

```
print(fun())
```

A)A

B

10

D

B)A

B

D

10

C)A

B

C

D

10

D)A

C

D

20

Q7. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)30

B)SyntaxError

C)TypeError

D)20

Q8. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

A)10

B)0

C)9

D)24

Q9. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

A)MyError

B)Something went wrong

C)Exception

D)TypeError

Q10. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)Module loaded

10

B)Module loaded

Module loaded

10

C)ImportError

D)10

10

Q11. What will be printed?

```
x = 10
```

```
def outer():  
    x = 20
```



```
def inner():  
    print(x)
```

```
x = 30  
inner()
```

outer()

- A)10
- B)30
- C)20
- D)NameError

Q12. What happens here?

```
def fun(a):  
    return a
```

print(fun())

- A)NameError
- B)TypeError
- C)None
- D)0

Q13. What will be the output?

```
import math
```

print(math.sqrt(25))

- A)5.0
- B)sqrt(25)
- C)5
- D)25

Q14. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

print(result)

- A)[2, 4]
- B)[2, 4, 6, 8, 10]
- C)[1, 3, 5]
- D)[4, 8]

Q15. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

- A)Nothing
- B)B
- C)A B
- D)A

Q16. What is the output?

```
f = lambda x: x + 5
```

print(f(10))

- A)5
- B>Error
- C)10
- D)15

Q17. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

print(outer())

- A)10
- B)UnboundLocalError
- C)15
- D)5

Q18. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```



return counter

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

A)0

1

0

2

B)1

2

1

3

C)1

1

1

1

D)1

2

3

4

Q19. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.imports

B)__module__

C)sys.path

D)sys.modules

Q20. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

A)30

B)-10

C)20

D)10

Q21. What does the following function return?

```
def fun():
```

```
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A)Hello

B>Error

C)Hello None

D)None

Q22. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

A)20

B)SyntaxError

C)30

D)TypeError

Q23. What will be printed?

```
class InvalidAgeError(Exception):
    pass
```

```
def check_age(age):
    if age < 18:
        raise InvalidAgeError("Age is too low")
    return "Allowed"
```

```
try:
    print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)
```

A)InvalidAgeError

15

B)InvalidAgeError

Age is too low

C)Nothing, because custom exceptions cannot be caught

D)Exception

Age is too low

Q24. What is the purpose of this statement?



if __name__ == "__main__":

- A)It identifies code intended to run when the file is executed directly
- B)It prevents syntax errors
- C)It creates the main function
- D)It prevents all imports

Q25. What will be the output?

x = 10

def outer():

x = 20

def inner():

global x

x = 30

inner()

print(x)

outer()

print(x)

A)20

30

B)20

10

C)30

30

D)30

20

Q26. What is the output?

def outer():

x = 10

def inner():

return x + 5

return inner

f = outer()

print(f())

A)None

B)15

C)10

D)NameError

Q27. What is the output?

try:

raise ValueError

except ValueError:

print("A")

raise

A)B

B)No Output

C)A, and execution ends normally

D)A, followed by propagation of the same exception

Q28. What will be printed?

def fun(x=[]):

x.append(10)

return x

a = fun()

b = fun()

print(a)

print(b)

A)[10]

[10, 10]

B>Error

C)[10, 10]

[10]

D)[10]

[10]

Q29. What will be printed?

try:

x = int("10")

except ValueError:

print("A")

else:

print("B")

finally:

print("C")

A)A C

B)B C

C)B

D)A

Q30. What is the major difference between the following?

import math

and



```
from math import sqrt
```

A)First provides the module namespace;
second imports sqrt directly

B)There is no difference

C)First imports only sqrt

D)Both import only sqrt

Q31. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)10

B)None

C)NameError

D)20

Q32. What will be the output?

```
x = 10
```

```
def fun():  
    print(x)  
    x = 20
```

```
fun()
```

A)NameError

B)UnboundLocalError

C)10

D)20

Q33. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)B C

B)A C

C)A

D)B

Q34. Why is

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

A)Exception is a superclass of
ZeroDivisionError

B)except blocks execute randomly

C)ZeroDivisionError occurs before
Exception

D)ZeroDivisionError is not an exception

Q35. What will be printed?

```
def fun():
```

```
    print("A")
```

```
    return
```

```
    print("B")
```

```
fun()
```

A)A

B)B

C)Nothing

D)A B

**Q36. What happens when this code
executes?**

```
def fun():
```

```
    print(variable)
```

```
    variable = 50
```

```
variable = 10
```

```
fun()
```

A)10

B)50

C)NameError

D)UnboundLocalError

Q37. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```



A)SyntaxError

B)A B

C)B

D)A

Q38. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

A)10

B)20

C)None

D)RuntimeError

Q39. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)Nothing

B)A and B

C)A

D)B

Q40. What will happen?

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal x  
        x = 20  
  
    inner()
```

outer()

A)NameError

B)20

C)10

D)SyntaxError

Q41. What will normally happen?

```
import math
```

```
import math
```

A)The module is normally loaded once and reused

B)The module is normally loaded and executed twice

C)SyntaxError

D)ImportError

Q42. What happens here?

```
from math import sqrt
```

print(sqrt(36))

A)NameError

B)6

C)36

D)6.0

Q43. What will happen?

```
def fun(a, b=10):  
    return a + b
```

print(fun(b=5, a=20))

A)15

B)25

C)TypeError

D)SyntaxError

Q44. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

A)The local file can shadow the intended standard-library module

B)Python will always reject it

C)random cannot be imported

D)Python automatically renames the local file

Q45. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```



print(fun(3))

A)None

B)3

C)5

D)6

Q46. What will happen?

def fun(n):

if n == 0:

return 1

return n * fun(n)

print(fun(3))

A)RecursionError

B)3

C)TypeError

D)6

Q47. What is the output?

def fun(x):

x += 10

return x

x = 5

y = fun(x)

print(x, y)

A)15 5

B)15 15

C)5 5

D)5 15

Q48. if test.py has

print("A")

if __name__ == "__main__":

print("B")

If test.py is executed directly, what will be printed?

A)A

B)Nothing

C)B

D)A followed by B

Q49. What will be the output?

def fun(x):

try:

if x < 0:

raise ValueError("negative")

return x

except ValueError:

return 100

finally:

print("cleanup")

print(fun(-5))

A)cleanup

100

B)negative

cleanup

C)100

cleanup

D)cleanup

-5

Q50. What is the output?

def fun(x):

return x * 2

a = fun

print(a(5))

A)10

B)func(5)

C>Error

D)5

